

Learning from Small Data: Tabular Foundation Models for Political Science

Marcel Neunhoeffer*

Job market paper

September 2026

Abstract

Political scientists often analyze a few hundred to a few thousand observations with mixed variable types, missing values and interactions that theory does not specify. Flexible learners need tuning, and at these sample sizes tuning competes with the analysis for the same data. Tabular foundation models are pretrained on millions of synthetic datasets and predict for a new dataset in a single pass, with nothing to tune. I evaluate one of them, TabPFN, as the prediction step in three typical applications. In a forecast of 603 Supreme Court cases decided after a published model was built, it is as accurate as that tuned model, and its probabilities score better than the published model's, even after that model is recalibrated. As the nuisance learner in double machine learning under nonlinear confounding, it reaches nominal coverage with the most accurate nuisance estimates of any learner tested, while tuned forests and boosting undercover. In multiple imputation, it recovers interactions that the imputer did not specify when the observed data identify them. I also report where it fails and provide open-source R software.

Keywords: tabular foundation models; in-context learning; double machine learning; multiple imputation; judicial politics; political methodology

*Institute for Employment Research (IAB) and LMU Munich. marcel@marcel-neunhoeffer.com. Software: <https://github.com/mneunhoe/tabfound>.

1 Introduction

Many relationships that political scientists study are interactive and nonlinear. Treatment effects vary across subgroups that were not anticipated (Green and Kern, 2012; Grimmer et al., 2017), risk factors for political violence matter in combination (Hill and Jones, 2014; Muchlinski et al., 2016), and theory rarely specifies the functional form (Montgomery and Olivella, 2018). At the same time, the datasets are usually small by machine-learning standards: a survey experiment with two thousand respondents, a panel of a few hundred countries, or the few hundred cases the Supreme Court decides in a decade, each with dozens of covariates of mixed type and with missing values. Regression handles data like these, but it asks the researcher to write down every interaction and nonlinearity in advance. Machine learning models like tree ensembles and Bayesian additive regression trees learn the functional form from the data (Montgomery and Olivella, 2018; Chipman et al., 2010). However, they come with hyperparameters that must be tuned and reported (Arnold et al., 2024), tuning that must be validated out of sample, and validation schemes that are easy to get wrong (Neunhoeffler and Sternberg, 2019). With a few hundred or a few thousand observations, each of these steps needs data that the analysis also needs.

Many analyses in political science contain a prediction step even when prediction is not their goal. A forecast of court decisions is a prediction problem. Double machine learning estimates a treatment effect after predicting the outcome and the treatment from the confounders (Chernozhukov et al., 2018). Multiple imputation draws missing values from a predictive distribution of the missing data given the observed data (Rubin, 1987; King et al., 2001). In each of these applications the quality of the prediction step decides whether the final result is valid, and in each of them the prediction step is typically fit to the same small dataset as the rest of the analysis.

In this paper, I evaluate tabular foundation models as the prediction step for these applications. A tabular foundation model is a neural network that its developers trained once on millions of synthetic datasets drawn from a wide range of data-generating processes. Applying it to a new dataset involves no training: the researcher’s data enter as context, and the model returns predictions for new observations in a single pass, which is called in-context learning. The model’s output can be read as an approximation to a posterior predictive distribution under the prior over data-generating processes that it was trained on (Müller et al., 2022). Because everything that is learned was learned in pretraining, there is nothing to tune. The model I study, TabPFN (Hollmann et al., 2023, 2025), handles mixed continuous and categorical features and missing values, and both its developers’ evaluation (Hollmann et al., 2025) and an independent benchmark (Erickson et al., 2025) find that it matches or beats tuned gradient-boosted trees on datasets of up to about 10,000 observations. This means that the model is designed for the sample sizes political scientists typically have.

I argue that tabular foundation models can be a sensible default for the prediction step in these applications, at small and moderate sample sizes, and I test this claim against baselines from political science. I first re-enter TabPFN in the simulation contest of Montgomery and Olivella (2018), which introduced tree-based models to political science. With nothing to tune, it has the lowest median test error on all three outcome surfaces and the lowest error in 299 of the 300 training

draws, including the additive surface on which every flexible model in the original contest lost to linear regression.

The first application is a forecast: Kaufman et al. (2019) predict Supreme Court decisions with tuned boosted decision trees from data on what the justices say at oral argument. On the paper’s own data, TabPFN is as accurate as the published model. I then forecast the 603 cases decided in the ten terms after the paper, each term from all earlier terms. TabPFN predicts 72.0 percent of these outcomes correctly and the published model 70.0 percent, a small though not statistically significant advantage. However, TabPFN’s log loss is lower in every one of the ten terms, and its predicted probabilities are better calibrated than the published model’s. Recalibrating the published model closes most of the gap in log loss, but it is one more step that has to be fitted. The comparison also yields a substantive observation: the published model’s most important feature, the relative number of lawyers who argue for each side, largely records whether the Solicitor General argues, and after 2015 this signal seems to carry more of the forecast than the justices’ questions.

In the second application, causal inference, double machine learning gives valid confidence intervals for a treatment effect only if the learners that estimate the nuisance functions are accurate enough, and simulation evidence shows that this has depended on careful tuning (Bach et al., 2024). I use a design in which the confounding follows the regime-switching and interactive surfaces of the Montgomery–Olivella contest. Across 500 replications, double machine learning with TabPFN as the nuisance learner covers the true effect in 95.0 percent of replications, with a bias of -0.002 . With a random forest at its defaults, it covers in 92.8 percent (Monte Carlo standard error 1.2). Tuning does not close the gap: a tuned forest and tuned boosting cover in 90.6 and 89.4 percent, clearly below nominal, and a stack of learners in 92.8 percent. Only a lasso on a dictionary of splines and pairwise interactions also reaches nominal coverage, with nuisance errors about twice those of TabPFN.

In the third application, multiple imputation, the model might have an advantage because an in-context learner is a conditional model of each variable given the others. It can therefore impute missing values without a model specification for each variable. On the imputation benchmark design of Kropko et al. (2014), extended to report confidence-interval coverage after pooling, TabPFN performs as well as mice where the established methods’ assumptions hold. When the analysis model contains an interaction that the imputer did not specify, its intervals for the interaction coefficient cover the truth in 95.0 and 94.0 percent of replications in two of three designs, where Amelia (Honaker et al., 2011) and mice (van Buuren and Groothuis-Oudshoorn, 2011) at their defaults almost never do. In the same two designs, imputation that is told about the interaction (Bartlett et al., 2015) covers in 96.0 and 95.5 percent. In the third design, where part of the data’s support is never observed, every method undercovers, TabPFN at 48.0 percent and the correctly specified imputation model at 82.5 percent.

These results come with several limitations. First, the current tabular foundation models are built for datasets of up to about 10,000 rows and 500 features, and their computational cost rises as the dataset they condition on grows. Second, double machine learning requires that the learners it

uses converge quickly enough to the truth. This condition has been proven for other model classes, such as deep neural networks (Farrell et al., 2021), but not yet for in-context learners (Nagler, 2023). The coverage results reported here therefore rest on simulations, not on theory. Third, better prediction does not solve the identification problem: a flexible learner cannot compensate for including controls that do not belong in the model (Hünermund et al., 2023). Finally, the prior used in pretraining was chosen by the model’s developers, and whether it suits political science data is an empirical question that each analysis must check.

The paper proceeds as follows. Section 2 places tabular foundation models among the flexible learners that political scientists already use. Section 3 explains how in-context learning works, with a small example. Section 4 reports the known-truth benchmark, and Sections 5 to 7 report the three applications. Section 8 summarizes the limits and concludes. The appendix describes the software (Appendix A) and gives additional results for each workflow. The R packages `tabfound` and `tabfound.infer` run the models and the estimators used here without Python (Neunhoeffler, 2026a,b).

2 Flexible Learners at Small Sample Sizes

2.1 The Cost of Tuning

Learning functional forms from data is an established practice in political methodology. Tree-based ensembles find interactions and nonlinearities that classical regression models require the researcher to anticipate (Montgomery and Olivella, 2018). Bayesian additive regression trees estimate heterogeneous treatment effects without a pre-specified list of moderators (Green and Kern, 2012; Chipman et al., 2010), and ensembles of different learners reduce the risk of committing to a single specification (Grimmer et al., 2017; van der Laan et al., 2007). Applications range from state repression (Hill and Jones, 2014) and civil war (Muchlinski et al., 2016) to Supreme Court forecasting (Kaufman et al., 2019). The field has also developed standards for what such models have to show: out-of-sample evaluation that is kept separate from tuning (Neunhoeffler and Sternberg, 2019) and caution about what we can learn from predictive accuracy (Cranmer and Desmarais, 2017).

However, these flexible learners come with hyperparameters (e.g., tree depths, learning rates, penalty weights, and stopping rules), and results can change considerably with their values. Default values are rarely defensible, tuning choices are rarely reported (Arnold et al., 2024), and the cross-validation needed for tuning is easy to get wrong: when the same cross-validation both tunes a model and estimates its error, the reported accuracy is too optimistic (Neunhoeffler and Sternberg, 2019). The temporal and hierarchical structure of much political science data adds further ways for information to leak from test to training data. At a few thousand observations this problem grows. Every observation used for validation is not available for fitting, and on small datasets tuned gradient-boosted trees no longer lead: a model that needs no tuning matches or beats them there (Hollmann et al., 2025; Erickson et al., 2025). This means that researchers currently accept either a specification burden (with regression) or a tuning burden (with flexible learners).

2.2 Tabular Foundation Models

A prior-data fitted network is a transformer that is trained once, offline, on millions of synthetic datasets drawn from a broad prior over data-generating processes (Müller et al., 2022). From these data-generating processes it learns a general mapping: given a labeled dataset and a new observation, it returns a predictive distribution for that observation. It applies this mapping to any new dataset in a single forward pass. TabPFN applies this idea to tabular data, with a prior built from structural causal models that favors simple functions, allows interactions, and includes irrelevant features (Hollmann et al., 2023). Its second generation added regression with a distributional output, native handling of categorical features and missing values, and accuracy that matches or exceeds tuned gradient-boosted trees on datasets of up to about 10,000 rows and 500 features (Hollmann et al., 2025). Later versions extend these limits (Grinsztajn et al., 2025, 2026; Jäger et al., 2026), and other groups have released related models (Qu et al., 2025; Zhang et al., 2025).

Three properties make these models particularly interesting for political science data. First, their advantage lies at small and moderate sample sizes. On a continuously updated benchmark, foundation models lead on small and medium tables, while boosted trees remain strong on large ones (Erickson et al., 2025). Second, their output is a distribution. For regression, the model returns a predictive density over a fine grid, and intervals, quantiles and simulated draws follow from it directly. Third, there is nothing to tune, so two researchers who run the same model version with the same seed on the same data obtain the same numbers. This removes a set of researcher choices that are rarely reported and that, as Arnold et al. (2024) show, can change which model appears best.

Currently, these models still lack a classical statistical theory. Their predictions can be understood as approximate Bayesian inference (Müller et al., 2022), and Nagler (2023) shows that the variance of these predictions shrinks toward zero as the number of observations the model conditions on grows. Their bias, however, depends on how well the pretraining prior fits the data at hand. It is also unknown whether these models converge to the truth quickly enough to meet the conditions that frequentist guarantees require when they are used inside larger estimators, such as double machine learning. Any claim that relies on these properties must therefore be checked empirically, using designs in which the true values are known. The analyses in this paper are built around such designs.

3 How In-Context Learning Works

3.1 Prediction without Training

A tabular foundation model never fits to the researcher’s data in the sense that it learns or updates parameters after seeing the data. Consider the familiar workflow first. To predict whether the petitioner wins a Supreme Court case with a logit, a researcher specifies a functional form, estimates the coefficients by maximum likelihood, and computes predicted probabilities from the fitted

coefficients. Every dataset produces its own fit, and the learning happens at analysis time. With a tabular foundation model, all learning happened once, before the researcher downloaded the model. What the researcher receives is a fixed function (the same file of network weights for every user) that maps a labeled dataset and a query observation to a predictive distribution for the query. In Section 5, for example, the model receives the 1,260 argued cases decided before the 2024 term together with the cases of that term, and it returns the probability that the petitioner wins each new case in one pass. The dataset plays the role that a prompt plays for a language model, which is why this mode of operation is called in-context learning.

This design has two practical implications. First, there are no hyperparameters to tune (no tree depth, no learning rate, no penalty), because no estimation runs on the researcher’s data. Second, conditioning on the data is instant, and the cost lies in prediction. The forward pass attends over the context rows and the query rows together, so its cost grows with both. On a laptop CPU, forecasting one Supreme Court term from all earlier terms takes one to two seconds.

3.2 The Pretraining Prior

The developers of tabular foundation models draw synthetic datasets from a generative prior (structural causal models with random graphs, random functional forms, random noise and random irrelevant features) and train the network on one task: given the labeled part of a synthetic dataset, predict its held-out outcomes (Müller et al., 2022; Hollmann et al., 2023). Over millions of such problems, the network learns how to learn from a small table. It does not learn a particular relationship, but a mapping from datasets to predictions.

In this paper, I use the Bayesian reading of this procedure. The pretraining distribution is a prior over data-generating processes, the researcher’s dataset is the evidence, and the model’s output approximates the posterior predictive distribution given both (Müller et al., 2022). Readers who use Bayesian models can think of TabPFN as an amortized posterior: the integration that Markov chain Monte Carlo would perform at analysis time was approximated once, for a very wide prior. Other readers need only note that the output for each query is a full probability distribution whose center and spread respond to the evidence in the context. The simulation approach of King et al. (2000) to quantities of interest carries over directly, because simulated outcomes can be drawn from this distribution.

3.3 A Toy Example

A small example illustrates how this works. I simulate 40 district elections in which the incumbent’s vote share depends on the district’s vote share in the previous election (drawn uniformly between 30 and 60) and on a binary scandal indicator, $y = 10 + 0.8 \cdot share_{t-1} - 8 \cdot scandal + \varepsilon$ with $\varepsilon \sim N(0, 3^2)$. TabPFN receives the 40 labeled districts as context and is queried for a new district with $share_{t-1} = 45$, first without and then with a scandal. It returns predictive distributions with medians of 45.2 and 38.5. The implied scandal penalty of 6.7 points is within sampling error of the

true 8 points at forty observations, and the 5/6 intervals¹ are about 8 points wide, which fits noise with a standard deviation of 3. I then add an interaction to the data-generating process, so that the penalty is 4 points larger in districts with $share_{t-1} < 40$. Without being told about it, the model now implies a penalty of 11.1 points for a district with $share_{t-1} = 35$ and of 7.4 points for a district at 45 (the true values are 12 and 8). This workflow produces no coefficients. That is a real cost: readers cannot look up a single number in a table that summarizes how an outcome changes with a covariate, and the model offers no compact equation to report or compare across studies. Much of what political scientists report, however, is computed from predictions, not read off coefficients. Predicted values, first differences and their uncertainty intervals are usually obtained by simulating outcomes for chosen covariate profiles (King et al., 2000). TabPFN supplies these quantities directly. In the example above, the scandal penalty is the difference between two predicted distributions, one for a district with a scandal and one without, and the same comparison at different values of $share_{t-1}$ reveals how the penalty varies across districts.

3.4 Limits of the Current Models

The released models are designed for datasets of up to about 10,000 rows and 500 features. Beyond that size, prediction quality declines and memory becomes a constraint (Hollmann et al., 2025). Because computation grows with the number of context and query rows, large test sets are scored in batches. The analyses in this paper run on a CPU alone, although the larger ones take several minutes. The models handle categorical variables and missing values natively but accept only tabular inputs, so text and images must first be converted into features. The lack of coefficients discussed above is less limiting than it may seem, because prediction-based interpretation tools such as permutation importance, partial dependence and scenario comparisons require only the ability to predict and apply without modification.

Tabular foundation models form a family, and its members differ in their pretraining prior, their output, the size of table they can take as context, and the license of their weights. All main results in this paper use TabPFN v2.5 with its default settings (Grinsztajn et al., 2025). The appendix reports results for other versions (TabPFN v2.6, v3 and v3.5) and other models (TabICL and Mitra) where I have them. The results for contests, forecasting and double machine learning hold across all the versions I ran.

4 A Known-Truth Benchmark

Montgomery and Olivella (2018) introduced tree-based models to political science with a simulation contest. It has three outcome surfaces of increasing difficulty: an additive surface, an interactive

¹I report central 5/6 intervals, bounded by the 1/12 and 11/12 quantiles of the predictive distribution, instead of the conventional 95 percent, as the election forecasts of Stoetzer et al. (2019) do. A 5/6 interval should miss the outcome about as often as a fair die shows a six, which gives readers an intuitive sense of its coverage. Narrower intervals also convey where outcomes are likely to fall, not where they are almost certain to fall, and they are easier to check against held-out data because misses occur often enough to count (Gelman, 2016). Under normal noise, a 5/6 interval spans about ± 1.38 standard deviations, so with $\sigma = 3$ its expected width is about 8.3 points.

surface, and a “complicated” surface with a regime switch. Each surface comes with 100 training datasets of 500 observations and 40 covariates, of which 30 are irrelevant, and the machine learning contestants are tuned over cross-validated grids. The design does not reward flexibility for its own sake: on the additive surface, every flexible model in the original contest lost to ordinary least squares. I regenerate their data draw for draw from the published code and seed and enter TabPFN with its default settings.

TabPFN has the lowest median test error on all three surfaces, and it has the lowest error of all models in 299 of the 300 training draws (Figure 1). On the additive surface, where OLS is correctly specified, TabPFN still beats it (median root mean squared error (RMSE) of 4.41 against 4.56). On the interactive surface its median error is 6.5 percent below that of tuned gradient boosting, and on the regime-switching surface, which was built to showcase trees, it is also slightly ahead of the best tuned ensemble (RMSE of 1,958 against 1,995). TabPFN v2.6 and v3 win 299 and 298 of the 300 draws. Running TabPFN on one training dataset, which means conditioning on its 500 observations and predicting the corresponding test set, takes about eight seconds on a laptop CPU.

These results suggest that the model adds flexibility only where the data support it and that it finds interactions without tuning. The benchmark also has the advantage of independence: Montgomery and Olivella designed it before tabular foundation models existed, so none of its choices could have been made to favor TabPFN. The surfaces of this contest return in Sections 6 and 7 as the confounding functions of the causal simulation and as data-generating processes of the imputation benchmark.

5 Forecasting Supreme Court Decisions

Forecasting the decisions of the U.S. Supreme Court is an established problem in judicial politics, and it has served as a test of whether statistical models can compete with legal experts (Martin et al., 2004) and how well they forecast out of sample (Katz et al., 2017). Oral argument is one of the few public signals available before a decision, and the justices’ questions seem to reveal their leanings: the side that receives more questions tends to lose (Epstein et al., 2010; Black et al., 2011). Kaufman et al. (2019) build on this finding and predict whether the petitioner wins with boosted decision trees. Their data are 697 cases decided in the 2005 to 2014 terms, described by 55 features: counts of what five of the justices say to each side at oral argument, the relative number of lawyers who argue for each side, and case characteristics from the Supreme Court Database (Spaeth et al., 2025). Their model predicts 74 percent of outcomes correctly, against 66 percent for the rule that the petitioner always wins. The authors recommend boosting for its “robustness to small sample sizes” and its treatment of “weakly predictive” covariates (Kaufman et al., 2019, 386). These are also the conditions for which tabular foundation models were built, which makes the paper a natural test case.

I proceed in three steps. First, I recover the published result. The archived predictions and my refit of the published configuration both predict 73.9 percent of outcomes correctly. Second,

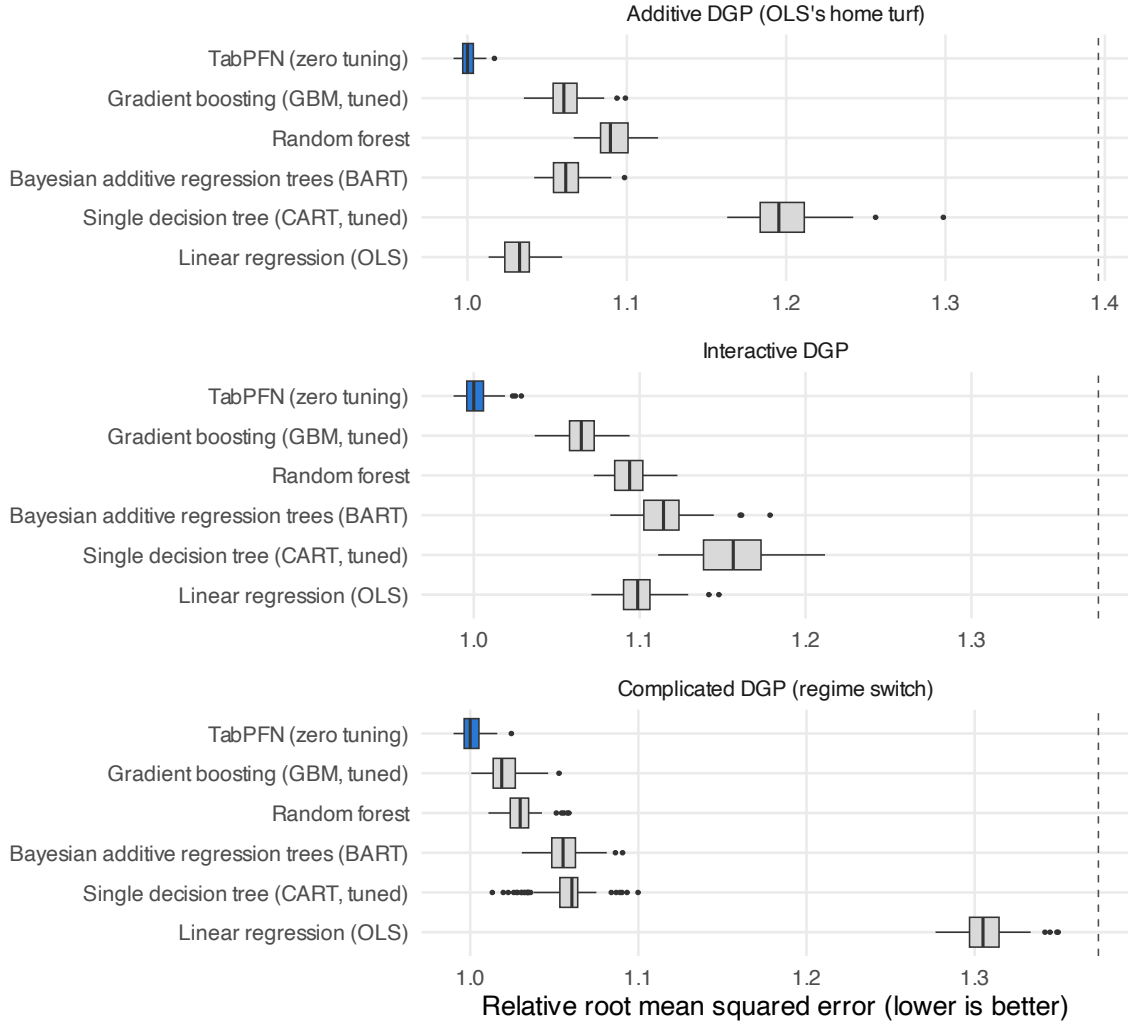


Figure 1. Relative Test Error in the Montgomery–Olivella Contest

Note: Test RMSE over 100 training draws per surface, relative to the best model’s median on that surface. Data regenerated draw for draw from the published code and seed of Montgomery and Olivella (2018) ($n = 500$ per draw, 40 covariates of which 30 are irrelevant, test sets of 3,000). CART and gradient boosting are tuned, and all comparison models use the settings of the original contest; TabPFN runs with its defaults. The dashed line marks the error of predicting the training mean.

I run TabPFN with its defaults on the paper’s own folds and features. It predicts 73.6 percent of outcomes correctly, and its differences to the refits with three seeds are all within 0.7 points, with confidence intervals that include zero (Table 1). The two models differ in their probabilities. TabPFN’s log loss² is 0.548. The published model’s log loss is 0.645, which is no better than a constant forecast at the base rate (0.644). Its scores rank the cases well, but they stay between 0.40 and 0.66, while petitioners win two thirds of the time.

Third, I forecast the cases decided after the Kaufman et al. (2019) paper. Each term from 2015 to 2024 is predicted from all earlier terms, which gives 603 test cases. I rebuilt the 55 features from the official transcripts with the original feature code, which reproduces the archived lawyer counts in 99 percent of cases, and I passed each justice’s features to the justice who holds that seat. The published configuration by Kaufman et al. (2019) was not re-tuned. I judge TabPFN to be as accurate as the published model if the 95 percent confidence interval of the difference in accuracy includes zero, and better if the interval excludes zero in its favor.

On the future cases, TabPFN predicts 72.0 percent of outcomes correctly and the published model 70.0 percent. The difference of 2.0 points has a 95 percent confidence interval from -0.3 to 4.5 , so the two models are equally accurate. TabPFN’s log loss (0.576 against 0.671) is lower in every one of the ten terms (Figure 2), and the published model again does worse than a constant forecast (0.634). The published model’s scores rank cases well but are poorly scaled. Logistic recalibration on out-of-fold scores within each training set repairs most of this: on the paper’s folds the recalibrated model’s log loss (0.542) is slightly below TabPFN’s (0.548), and on the future terms it falls to 0.594, still above TabPFN’s 0.576. Recalibration is one more fitted and validated step. TabPFN’s probabilities need no such step. Both models lose accuracy compared with the paper’s folds, the published model more than TabPFN (3.9 against 1.6 points). The forecast is also available on the day of argument. Without the database codes, which are assigned after the decision, TabPFN still predicts 71.0 percent of the outcomes correctly. TabPFN v3.5, TabICL and Mitra predict between 71.0 and 71.8 percent correctly, with log losses of at most 0.579 (Table B1).

The comparison also shows what the absence of tuning means when the features change. On 18 features that do not depend on which justices sit, TabPFN predicts 71.1 percent of future outcomes correctly. The published configuration falls to 63.5 percent, below the rule that the petitioner always wins (67.2 percent). Here TabPFN is better by the decision rule (a difference of 7.6 points, with a 95 percent confidence interval from 4.1 to 11.3). With only weak features, its boosting runs for many more rounds and seems to fit noise (Appendix B), while TabPFN’s probabilities stay calibrated and follow the one strong signal.

The strongest signal in the data is the relative number of lawyers. In the replication archive it is labeled `amicus`, and it is the most important feature of the published model. When I code the

²Log loss is the average negative log probability that a model assigns to the outcomes that actually occurred: $-\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$, where y_i is the observed outcome (0 or 1) and $\hat{p}_i$ is the predicted probability that $y_i = 1$. Lower values are better. A model that always predicts the correct outcome with certainty scores 0, and a model that assigns probability 0.5 to every case scores $\log 2 \approx 0.693$. Unlike accuracy, log loss rewards well-calibrated probabilities and penalizes confident mistakes heavily. A log loss of 0.548 means that, on average (as a geometric mean), the model assigned a probability of about $e^{-0.548} \approx 0.58$ to the outcome that occurred.

Table 1. Forecasting Supreme Court Outcomes with the Published Model and with TabPFN

Model	Paper’s folds OT2005–14		Future terms OT2015–24		Future terms, argument-day features	
	Accuracy	Log loss	Accuracy	Log loss	Accuracy	Log loss
Petitioner always wins	65.9	0.644	67.2	0.634	67.2	0.634
KKS, as archived	73.9	—	—	—	—	—
KKS, refit	73.9	0.645	70.0	0.671	69.7	0.673
KKS, refit and recalibrated	73.2	0.542	69.5	0.594	—	—
AdaBoost, software defaults	72.0	0.615	67.3	0.628	68.3	0.629
TabPFN, no tuning	73.6	0.548	72.0	0.576	71.0	0.582

Note: Accuracy in percent (cutoff 0.5). Log loss: lower is better; a constant forecast at the training-period petitioner-win rate scores 0.644 on the paper’s folds and 0.634 on the future terms. Paper’s folds: the 697 decided cases, ten folds and 55 features of Kaufman et al. (2019). Future terms: each term OT2015–OT2024 forecast from all earlier terms (603 cases), with the features rebuilt from the official transcripts by the original feature code and each justice’s features assigned to the holder of that seat. Argument-day features: the 47 oral-argument features without the Supreme Court Database codes, which are assigned after the decision. KKS: the published configuration, not re-tuned, with probabilities averaged over three seeds. Recalibrated: logistic recalibration on out-of-fold scores within each training set. Future terms, TabPFN minus KKS: accuracy +2.0 points (95% bootstrap CI -0.3 to 4.5), log loss -0.095 (-0.129 to -0.057).

side of the United States from the transcripts’ lists of appearances, the United States argues on the side with more lawyers in 90 percent of the cases in which the counts differ. This means that the feature largely records whether the Solicitor General argues and on which side, a well-documented predictor of Supreme Court outcomes (Bailey et al., 2005; Black and Owens, 2012). An exploratory analysis in Appendix B suggests that this signal accounts for most of the improvement of TabPFN’s seat-free forecast over a constant forecast after 2015, while the justices’ questioning adds little.³

Two limits apply. The published model is more accurate in two of the ten terms, the two models tie in two, and with about 60 cases per term, differences between single terms are mostly noise. The result is therefore a tie in accuracy. What TabPFN adds is that it matches a tuned, published model on cases decided after that model was built, with better probabilities and no tuning.

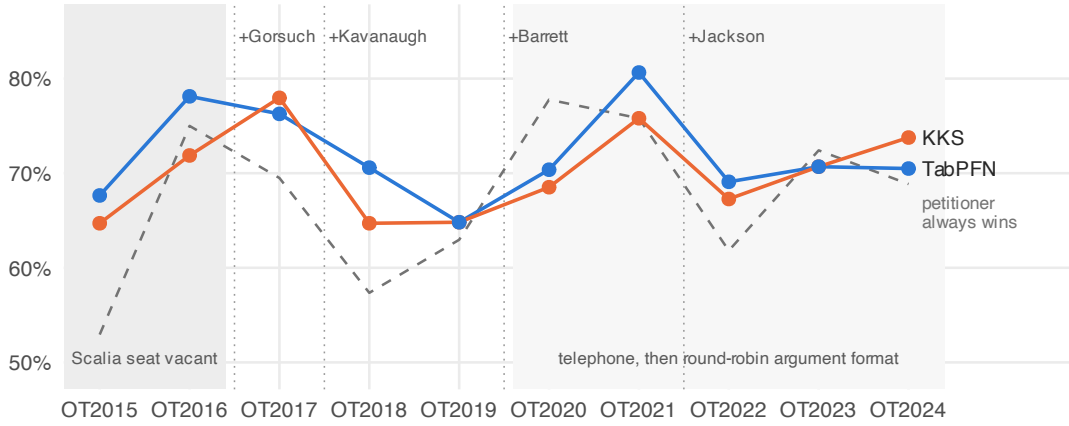
6 Treatment Effects with Double Machine Learning

6.1 Why the Nuisance Learner Matters

Political scientists often want to report effects with confidence intervals, and the most widely used framework for obtaining valid inference from flexible predictions is double or debiased machine

³In the authors’ replication archive, the model’s configuration and features were selected using the same cross-validation folds that produce the reported accuracy, so the published figure may be somewhat optimistic (Appendix B). This was standard practice at the time, and it is visible only because the authors made their code available. The forward test is unaffected, because the published configuration was chosen using data from the 2005 to 2014 terms only.

A Accuracy (share of cases predicted correctly)



B Log loss (lower is better)

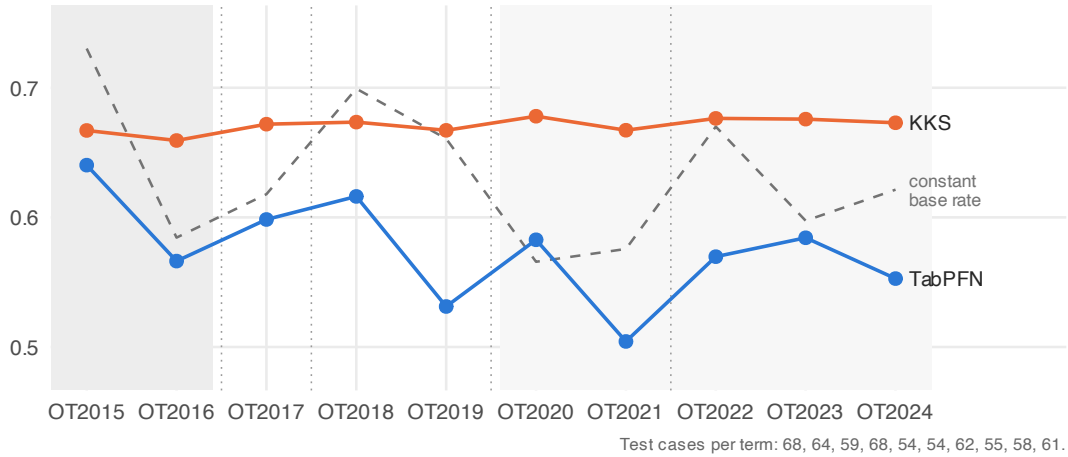


Figure 2. Forecasting Each Future Term of the Supreme Court

Note: Each term OT2015–OT2024 is forecast from all earlier terms (OT2005 onward), 603 cases. Both models receive the same 55 features of Kaufman et al. (2019), rebuilt from the official transcripts with the original feature code; each justice’s features pass to the holder of that seat. KKS: the published configuration, not re-tuned (average of three seeds). TabPFN: v2.5 with its defaults. Dashed lines: petitioner always wins (accuracy) and a constant forecast at the training-period rate (log loss). Dotted lines mark the arrival of new justices; shading marks the Scalia vacancy and the changed argument format.

learning (DML) (Chernozhukov et al., 2018; Ahrens et al., 2025). In the partially linear model

$$Y = \theta D + g(X) + \varepsilon, \quad D = m(X) + v, \quad (1)$$

a learner estimates the two nuisance functions, the outcome regression $\ell(X) = E[Y | X]$ and the treatment equation $m(X) = E[D | X]$. The effect θ is then estimated by regressing the outcome residual $Y - \hat{\ell}(X)$ on the treatment residual $D - \hat{m}(X)$. This score is Neyman-orthogonal, which means that first-order errors in the nuisance estimates do not affect the estimate of θ . With cross-fitting, no observation is residualized by a model that was fit to it. Under conditions on the accuracy of the nuisance estimates, the estimator is root- n consistent and asymptotically normal (Chernozhukov et al., 2018). The condition is a product: the errors of $\hat{\ell}$ and $\hat{m}$ have to shrink fast enough together, so a more accurate estimate of one nuisance relaxes the requirement on the other (Kennedy, 2024). The framework is available in standard software (Bach et al., 2022).

The guarantee is conditional, and the condition can be demanding at small samples. In the most systematic simulation study to date, Bach et al. (2024) find that nuisance learners at default or poorly tuned settings can bias DML estimates enough to invalidate their confidence intervals. They also find that the quality of the causal estimate follows the out-of-sample accuracy of the nuisance fits, and that the way tuning is done (on the full sample, on a held-out split, or within folds) has consequences of its own. Grimmer et al. (2017, 414) make the same argument for heterogeneous treatment effects, which is why their ensemble weights learners by cross-validated prediction error. Machlanski et al. (2025) also find that hyperparameter choices are a first-order determinant of the accuracy of causal estimates. Stacking many learners inside cross-fitting reduces the risk of a poor choice (Ahrens et al., 2025), but it multiplies tuning and computation. A tabular foundation model fits these requirements well. It is most accurate relative to tuned learners at the sample sizes where tuning is least affordable, and it needs no tuning at all. TabPFN can already be used with standard DML software, and a documented example exists (DoubleML developers, 2025). However, a working implementation does not show that the resulting confidence intervals are valid. To my knowledge, there is no known-truth evidence yet on whether DML with foundation-model nuisances reaches nominal coverage under nonlinear confounding at political-science sample sizes.

Another line of research builds the causal estimator into the foundation model itself. Models pretrained on interventional data return treatment effects in a single forward pass (Robertson et al., 2025; Balazadeh et al., 2025; Ma et al., 2025). I do not use them here, because Melnychuk et al. (2026) show that such estimators of the average treatment effect can carry a prior-induced confounding bias, because the prior is not asymptotically overwritten by the data, and that a one-step correction of the posterior restores frequentist consistency. The orthogonal score of DML plays a similar role. I therefore keep identification and the frequentist guarantees in the research design and the orthogonal score, and I use the foundation model only for the nuisance functions.

6.2 Design

The simulation places the confounding in nonlinear functions that require a flexible learner to capture. I use the regime-switching surface of Section 4 as the confounding function g in equation (1) and the interactive surface as the treatment equation m . Both surfaces depend on the same three of the 40 covariates, so the confounding is strong (the two functions correlate at 0.44), nonlinear and non-monotone. Both surfaces are standardized, the noise terms are standard normal, the true effect is $\theta = 0.5$, and each of the 500 replications has $n = 500$ observations.

I compare four estimators that use no tuning. PLUG-IN fits TabPFN to the outcome on the full sample without the treatment and regresses the outcome residual on the raw treatment, which is what a naive “control with machine learning” workflow produces. It has neither an orthogonal score nor cross-fitting. OLS-FULL regresses the outcome on the treatment and all 40 covariates linearly. DML with a random forest uses a forest at its package defaults for both nuisances, and DML with TabPFN uses TabPFN at its defaults. All DML estimators use five folds and the partialling-out score.

Because careful analysts tune their learners, the comparison includes four DML estimators whose nuisance learners are tuned inside each cross-fitting fold, on the same 500 datasets: a random forest tuned over the number of candidate variables and the minimum node size by out-of-bag error, a lasso on a dictionary of natural splines for every continuous covariate and all pairwise products of the covariates, with the penalty chosen by cross-validation, gradient boosting, with depth and number of rounds chosen by cross-validation, and a stack of these three learners and OLS, with non-negative weights chosen by cross-validation (van der Laan et al., 2007), as in the ensemble of Grimmer et al. (2017). I include the details in Appendix C. For every estimator, I report bias, the standard deviation and RMSE of the estimates, coverage with its Monte Carlo standard error, the mean width of the confidence interval, and the accuracy of both nuisance estimates against the true functions.

6.3 Results

Table 2 reads from failure to nominal coverage. PLUG-IN is biased by nearly half the true effect (-0.23), and its intervals contain the truth in none of the 500 replications. Flexible prediction used as a control strategy, without the orthogonal score and without cross-fitting, therefore gives a precise estimate of the wrong quantity. OLS-FULL is nearly unbiased here (-0.020) but covers in only 92.0 percent of the replications, because linear controls cannot absorb curved confounding and its standard errors do not reflect this. DML with a default random forest has a similar bias (-0.017) and covers in 92.8 percent. Its estimate of the treatment equation misses the truth by a root mean squared error of 0.54. DML with TabPFN covers in 95.0 percent (Monte Carlo standard error 1.0) with a bias of -0.002 . TabPFN also recovers the two confounding functions more accurately. Its predictions of the treatment from the covariates deviate from the true treatment equation by 0.25 (root mean squared error), and its predictions of the outcome from the covariates deviate from the true outcome equation by 0.30. Both errors are about half as large as those of the forest. DML

Table 2. Estimates of a Treatment Effect under Nonlinear Confounding

Estimator	Bias	SD	RMSE	Coverage (MCSE)	CI width	$\hat{m}$	$\hat{\ell}$
<i>No tuning</i>							
PLUG-IN (TabPFN)	−0.229	0.036	0.232	0.0 (0.0)	0.127	—	—
OLS-FULL	−0.020	0.057	0.061	92.0 (1.2)	0.216	—	—
DML, random forest (defaults)	−0.017	0.042	0.045	92.8 (1.2)	0.170	0.54	0.43
DML, TabPFN (defaults)	−0.002	0.045	0.045	95.0 (1.0)	0.178	0.25	0.30
<i>Tuned inside each cross-fitting fold</i>							
DML, random forest (tuned)	−0.023	0.044	0.050	90.6 (1.3)	0.172	0.47	0.39
DML, lasso (splines, interactions)	0.010	0.048	0.049	95.2 (1.0)	0.183	0.50	0.59
DML, boosting (tuned)	−0.029	0.043	0.052	89.4 (1.4)	0.173	0.49	0.41
DML, stack of four learners	−0.015	0.044	0.047	92.8 (1.2)	0.175	0.43	0.39

Note: Partially linear model with Montgomery–Olivella confounding (true effect 0.5, $n = 500$, 40 covariates, 500 replications; every estimator sees the same 500 datasets). Bias, SD (standard deviation of the estimates) and RMSE refer to the effect estimate. Coverage: share of nominal 95 percent confidence intervals that contain the true effect, in percent, with its Monte Carlo standard error. The Monte Carlo standard error of the bias is $SD/\sqrt{500}$, between 0.002 and 0.003 for every estimator. CI width: mean width of the interval. $\hat{m}$, $\hat{\ell}$: root mean squared error of the cross-fitted estimates of the treatment equation $E[D | X]$ and the outcome equation $E[Y | X]$ against the truth. All DML estimators use five-fold cross-fitting and the partialling-out score. Tuned learners are tuned inside each training fold (Appendix C).

yields a reliable effect estimate when the product of the two errors is small, and halving each error cuts their product to about a quarter.

The second part of the table shows that the gap persists after tuning. The tuned random forest estimates the treatment equation more accurately than the default forest (0.47 against 0.54). Its effect estimates are more biased (−0.023), and its intervals cover in 90.6 percent. Tuned boosting covers in 89.4 percent and the stack in 92.8 percent, with biases of −0.029 and −0.015. The stack puts most of its weight on the random forest in the outcome equation (0.62 on average) and so inherits much of its error. Among the tuned learners, the lasso alone reaches nominal coverage, at 95.2 percent. Its dictionary contains the pairwise products of the covariates through which the confounding works, so a researcher who chooses this dictionary has already anticipated much of the functional form. Its nuisance errors are about twice those of TabPFN (0.50 and 0.59 against 0.25 and 0.30), and its intervals are slightly wider (0.183 against 0.178). Both biases are small relative to the true effect of 0.5. TabPFN, run with its default settings, ties the default random forest for the smallest RMSE of the effect estimate (0.045), with a smaller bias, and is more accurate than every tuned learner (0.047 to 0.052).

In two easier companion designs, the canonical partially linear and interactive designs of Chernozhukov et al. (2018), the confounding is close to linear and all adjustment estimators, including OLS-FULL, cover near the nominal rate. DML with TabPFN also does, at 94.2 and 96.7 percent (Appendix C). With TabPFN v2.6 and v3 instead of v2.5, DML covers at 94.4 and 94.6 percent in the main design. Across the three designs, the TabPFN estimator was evaluated in 1,300 replications.

6.4 What the Evidence Does Not Show

These are empirical coverage results on known data-generating processes at a political-science sample size, and a formal guarantee is still missing. DML produces valid confidence intervals only if the learner’s prediction errors shrink quickly enough as the sample grows. This has been proven for other model classes, for example deep neural networks (Farrell et al., 2021). For prior-data fitted networks, current results show that the variance of the predictions shrinks as the sample grows (Nagler, 2023), and whether the bias shrinks quickly enough remains an open question. A researcher whose application departs far from these designs should check coverage in a simulation that resembles the problem. The `dml_plr()` and `dml_irm()` functions in `tabfound.infer` accept any learner through a two-line interface, and the replication materials contain the simulation code, so such a check requires little additional work.

7 Multiple Imputation

Multiple imputation needs a model for the missing values given the observed ones (Rubin, 1987), and political science has two established ways to build one (Kropko et al., 2014). Amelia draws imputations from a joint multivariate normal model of all variables (King et al., 2001; Honaker et al., 2011), and mice models each incomplete variable conditionally on the others, one regression at a time (van Buuren and Groothuis-Oudshoorn, 2011). Kropko et al. (2014) showed that the conditional approach is more accurate for categorical variables and as accurate for continuous variables. An in-context learner is a conditional model of each variable given the others, and it also has a posterior predictive distribution to draw from. This means that the imputer does not have to specify a model for each variable. The TabPFN imputations are chained draws from its predictive distribution (with its classification head for categorical variables), and the estimates are pooled with Rubin’s rules.

Amelia assumes that all variables are jointly multivariate normal, and mice fits a separate regression model with a researcher-specified form for each variable. TabPFN leaves the functional form to be learned from the data. This flexibility can be important when the analysis model contains structure, such as an interaction, that the imputation model does not represent. The benchmark follows Kropko et al. (2014) in its variable types, missingness mechanisms, and real-data exercise on the 2008 American National Election Study (ANES). I extend their design in two ways. First, I report whether the confidence intervals of the downstream analysis, after the imputed datasets are combined with Rubin’s rules, contain the true values at the nominal rate. Second, I add simulation scenarios in which the analysis model includes interactions or uses the Montgomery–Olivella surfaces as its outcome functions. In each replication, all methods impute the same incomplete dataset ($m = 10$ imputations). Appendix D describes each scenario.

Figure 3 summarizes the benchmark. Where Amelia’s assumptions hold (multivariate normal data with missingness completely at random or at random), the three methods cover at 94 percent or more. When missingness depends on the missing values themselves, all three fail together, as any method that assumes ignorable missingness must. In both cases the benchmark reproduces the

established findings. I want to highlight three further results.

The interaction scenarios. In three designs the analysis model is a linear regression with an interaction. In the first, only the outcome is missing, at random given the binary moderator. In the second and third, the continuous and the binary covariate are missing, completely at random in one design and depending on the outcome in the other. Amelia and mice work as designed here, but their imputation models do not contain the interaction, so the pooled intervals for its coefficient cover the truth in none or almost none of the replications (0.5 percent or less in the first two designs; Table 3). A user would not see this failure, and analysis models in applied work often contain interactions. TabPFN, which is not told about the interaction, covers its coefficient in 95.0 and 94.0 percent of the replications in the first two designs.

I also compare three further imputation methods on the same datasets, each representing a standard way of handling a known or suspected interaction. The first adds the interaction term to the dataset and imputes it like any other variable (von Hippel, 2009), here with mice’s default predictive mean matching. The second is mice with random forests in place of regression models for each variable, which can detect interactions without being told about them (Doove et al., 2014). The third is substantive-model-compatible imputation, which imputes each covariate from a model consistent with the analysis model, including its interaction (Bartlett et al., 2015). Because it builds in the correct analysis model, it is the benchmark for these designs, and its 95 percent confidence intervals contain the true value in 96.0 and 95.5 percent of replications in the first two designs. TabPFN comes within two percentage points of it without being told the analysis model. Imputing the interaction term like any other variable yields 95.5 percent coverage when only the outcome is missing and 0 percent when the variables in the interaction are missing. With a normal imputation model this approach is consistent when covariates are missing completely at random (Seaman et al., 2012), so the failure here likely comes from predictive mean matching, which Seaman et al. (2012) also find to be more biased. Random-forest mice reaches 16.0 and 0 percent coverage.

In the third design all methods undercover. The missingness mechanism deletes both covariates in every row whose outcome falls below -1 (16 percent of the rows), so part of the joint distribution is never observed together. Substantive-model-compatible imputation, which extrapolates with the correct parametric model, comes closest (82.5 percent, with a bias of -0.03). Random-forest mice covers in 79.5 percent but with a bias of 0.19 and intervals more than five times as wide. TabPFN covers in 48.0 percent with a bias of 0.39, and mice at its defaults in 46.5 percent. This means that where the observed data do not identify the interaction, knowing the analysis model is the only way to recover it. Where they do, TabPFN matches the correctly specified model without that knowledge.

The categorical and real-data scenarios. These scenarios reproduce the advantage of mice over Amelia that Kropko et al. (2014) found, and TabPFN does as well as mice. With a binary outcome, Amelia fails to produce estimates in all 200 replications of each scenario, which reflects a known fragility of the joint-normal approach. In the two cells with less missingness, TabPFN’s mean coverage (0.94 in both) is close to mice’s (0.96 and 0.98), with intervals 8 and 30 percent narrower. In

Table 3. The Interaction Coefficient after Multiple Imputation

Imputation	Outcome missing, MAR			Covariates missing, MCAR			Covariates missing, MAR		
	Bias	Cov.	Width	Bias	Cov.	Width	Bias	Cov.	Width
<i>Not told about the interaction</i>									
Amelia	-0.308	0.0	0.153	0.688	0.0	0.366	0.930	0.0	0.538
mice	-0.297	0.0	0.178	0.822	0.5	0.851	0.941	46.5	1.901
mice, random forests	-0.220	16.0	0.307	0.285	0.0	0.260	0.186	79.5	0.473
TabPFN	-0.001	95.0	0.101	0.002	94.0	0.067	0.386	48.0	0.762
<i>Told about the interaction</i>									
mice, product term	0.000	95.5	0.118	0.715	0.0	0.365	0.980	0.0	0.498
SMC-FCS	0.001	96.0	0.106	-0.001	95.5	0.060	-0.026	82.5	0.088

Note: Pooled estimate of the interaction coefficient in the analysis model (200 replications per cell, $n = 1,000$, $m = 10$ imputations, Rubin’s rules; every method imputes the same incomplete datasets). Bias: mean pooled estimate minus the true coefficient (1 in the first cell, -2 in the others). Cov.: coverage of the nominal 95 percent interval, in percent (Monte Carlo standard error about 1.5 points near 95 percent). Width: mean interval width. mice, product term: the product term added to the data and imputed like any other variable (von Hippel, 2009). SMC-FCS: substantive-model-compatible imputation, `smcfcs` with the analysis model, interaction included (Bartlett et al., 2015); where only the outcome is missing it reduces to Bayesian linear-regression imputation with the product term.

the cell with the heaviest missingness, TabPFN’s mean coverage drops to 0.87, while mice holds 0.96 with intervals nearly twice as wide. This suggests that TabPFN’s imputations can understate the between-imputation variance when much of the data is missing. On the ANES exercise, TabPFN’s pooled estimates are as close to the full-data estimates as mice’s (1.12 against 1.13 half-widths of the full-data interval), and Amelia’s are further away (2.01).

The 40-variable scenarios. The two most demanding scenarios use the interactive and regime-switching surfaces of Section 4, with 39 incomplete variables that are each about 19 percent missing. TabPFN’s coverage is as low as mice’s (0.21 against 0.20 in one cell and 0.75 against 0.69 in the other), and Amelia does not complete the computation. However, the same analysis on the complete data does no better (0.19 and 0.66). The analysis model is a linear regression with main effects only, so it misses most true coefficients of these surfaces even without missing data. These two scenarios therefore show a limit of the analysis model, which no imputation method can repair.

8 Limits and Conclusion

In this paper, I evaluated a tabular foundation model as the prediction step in three applications that political scientists often use: forecasting, estimating treatment effects with double machine learning, and multiple imputation. In each workflow, TabPFN with its defaults did as well as or better than the field’s standard tools in most designs, at the sample sizes the field typically has. In each workflow I also found conditions under which it does worse. In this section, I summarize the limits of the models and of the evidence.

Data size and computation. The released models are designed for up to roughly 10,000 observations and 500 features. For larger datasets, prediction quality degrades, memory becomes a

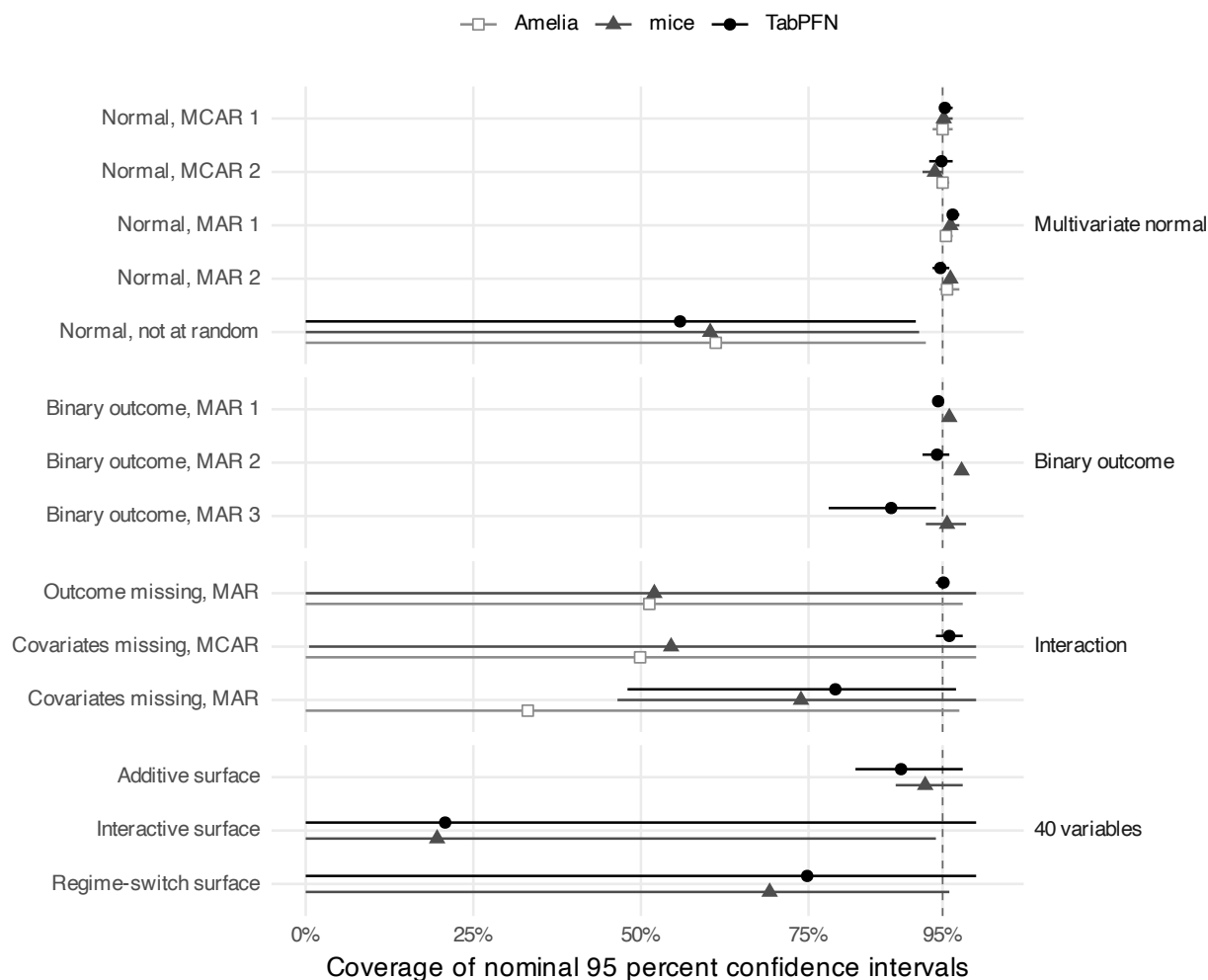


Figure 3. Coverage after Multiple Imputation, by Benchmark Cell and Method

Note: Coverage of nominal 95 percent confidence intervals of the analysis model's coefficients after multiple imputation ($m = 10$, Rubin's rules; every method imputes the same incomplete datasets; 200 replications per scenario, 50 in the 40-variable scenarios). Points are means over the coefficients; lines span the worst to the best coefficient. Amelia did not complete the binary-outcome and 40-variable scenarios (Table D1). The dashed line marks 95 percent.

constraint, and boosted trees remain the sensible default (Grinsztajn et al., 2022; Erickson et al., 2025). Because the forward pass uses the context and the query rows together, its cost grows with both, and a fitted tree ensemble scores new observations faster. For the analyses in this paper the cost was never a constraint, but a bootstrap with thousands of refits, or a simulation with millions of predictions, can be expensive. More recent successor models extend this range (Grinsztajn et al., 2026; Jäger et al., 2026).

Theory. The coverage results for double machine learning are empirical, from known data-generating processes, and the conditions on convergence rates that would turn them into a guarantee are open for in-context learners (Nagler, 2023). I think this is the most important open question for using these models in inference.

Calibration. A model is well calibrated when its predicted probabilities match observed frequencies: among cases to which it assigns a 70 percent chance that the petitioner wins, the petitioner should win about 70 percent of the time. The expected calibration error summarizes departures from this ideal. It groups cases by their predicted probability, compares the average prediction in each group with the share of petitioner wins, and averages the gaps, weighting each group by its number of cases. TabPFN’s error was 0.050 on the cross-validation folds of the original study and 0.064 on the later terms, meaning its probabilities were off by about five to six percentage points on average. The published model’s errors were 0.19 and 0.17. TabPFN’s probabilities were thus much better calibrated than those of the published model as fitted, although their calibration worsened somewhat on new data. Logistic recalibration of the published model removes this difference, at the cost of one more fitted step. Researchers should therefore check calibration in each analysis. A reliability plot, which draws the observed share of outcomes against the predicted probability for each group, requires only a few lines of code.

Structured data. The Supreme Court forecast respects the temporal order of the data, but I did not study clustered or hierarchical dependence systematically. The DML functions do not yet provide cluster-robust standard errors, and cross-fitting with randomly assigned folds ignores dependence between observations, so information can leak across folds in clustered or temporally ordered data. Researchers with time-series cross-section or multilevel data should treat the coverage results as encouraging but not as sufficient.

Identification and the analysis model. Nothing in this paper relaxes unconfoundedness, overlap, or the exclusion of post-treatment variables. DML assumes the design, and the model only estimates under it. A flexible learner does not protect against bad controls (Hünemund et al., 2023), and it may fit them more accurately than a weak learner would, so a design error can produce a narrow interval around a wrong value. Likewise, a better imputation model cannot repair a misspecified analysis model, as the 40-variable scenarios of Section 7 show, and it cannot recover structure that the observed data do not contain, as the third interaction design shows.

The pretraining prior. The model’s developers chose the prior for general tabular prediction, and researchers can neither inspect nor modify it, much as users of large language models cannot inspect the data those models were trained on. In this paper, I treat the prior’s suitability for political

science data as an empirical question, and researchers should do the same as new versions appear. Results can also depend on details beyond the prior. In the Supreme Court forecast, TabPFN’s accuracy on the seat-free features shifted by about one percentage point when the order of the columns changed (Appendix B). Researchers should therefore report the model version, the random seed and the column order, and fix all three in their replication materials.

From these results, I draw five recommendations for applied work. First, use tabular foundation models inside their size range and say so. Second, evaluate them out of sample with splits that respect the structure of the data, as if they had hyperparameters. Third, check calibration in every analysis. Fourth, for causal claims, use the orthogonalized estimators and report the nuisance diagnostics, and never use the plug-in shortcut, whose intervals covered the truth in none of the 500 replications. Fifth, report where established methods perform better, and record the model version, random seeds and column order in the replication materials.

The same approach can be useful for many questions in political science that come with small tables. Forecasts of conflict onset, heterogeneous effects in survey experiments, comparative panels with a few dozen countries, and models of judicial behavior all combine a few hundred to a few thousand observations with covariates of mixed type and with interactions that theory does not specify. In each of these applications, good average predictions are only the first requirement, and the question that matters is whether the analysis that uses them remains valid. I hope that the standard applied here, checking a foundation model on problems where the answer is known before relying on it where the answer is unknown, becomes the standard for using such models in political science.

Acknowledgements

The author made use of Claude Opus 5 and Claude Opus 5.5, as well as other Claude models (Anthropic), accessed through claude.ai / Claude Code between July 2026 and September 2026. The tools were used without modification to brainstorm ideas and research designs, to draft and revise text, and to write and run code for the analyses. The author evaluated all suggestions, reviewed and edited all text, verified every reference against the original source, and checked all code and results. The author is entirely responsible for the scientific content of the paper.

References

- Ahrens, A., V. Chernozhukov, C. Hansen, D. Kozbur, M. Schaffer, and T. Wiemann (2025). An introduction to double/debiased machine learning. [arXiv:2504.08324](https://arxiv.org/abs/2504.08324).
- Arnold, C., L. Biedebach, A. Küpfer, and M. Neunhoeffler (2024). The role of hyperparameters in machine learning models and how to tune them. *Political Science Research and Methods* 12(4), 841–848.
- Bach, P., V. Chernozhukov, M. S. Kurz, and M. Spindler (2022). DoubleML – an object-oriented implementation of double machine learning in Python. *Journal of Machine Learning Research* 23(53), 1–6.
- Bach, P., O. Schacht, V. Chernozhukov, S. Klaassen, and M. Spindler (2024). Hyperparameter tuning for causal inference with double machine learning: A simulation study. In *Proceedings of the Third Conference on Causal Learning and Reasoning*, Volume 236 of *Proceedings of Machine Learning Research*, pp. 1065–1117. PMLR.
- Bailey, M. A., B. Kamoie, and F. Maltzman (2005). Signals from the tenth justice: The political role of the Solicitor General in Supreme Court decision making. *American Journal of Political Science* 49(1), 72–85.
- Balazadeh, V., H. Kamkari, V. Thomas, B. Li, J. Ma, J. C. Cresswell, and R. G. Krishnan (2025). CausalPFN: Amortized causal effect estimation via in-context learning. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*.
- Bartlett, J. W., S. R. Seaman, I. R. White, and J. R. Carpenter (2015). Multiple imputation of covariates by fully conditional specification: Accommodating the substantive model. *Statistical Methods in Medical Research* 24(4), 462–487.
- Black, R. C. and R. J. Owens (2012). *The Solicitor General and the United States Supreme Court: Executive Branch Influence and Judicial Decisions*. Cambridge: Cambridge University Press.
- Black, R. C., S. A. Treul, T. R. Johnson, and J. Goldman (2011). Emotions, oral arguments, and Supreme Court decision making. *The Journal of Politics* 73(2), 572–581.
- Chernozhukov, V., D. Chetverikov, M. Demirer, E. Duflo, C. Hansen, W. Newey, and J. Robins (2018). Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal* 21(1), C1–C68.
- Chipman, H. A., E. I. George, and R. E. McCulloch (2010). BART: Bayesian additive regression trees. *The Annals of Applied Statistics* 4(1), 266–298.
- Cranmer, S. J. and B. A. Desmarais (2017). What can we learn from predictive modeling? *Political Analysis* 25(2), 145–166.
- Doove, L. L., S. van Buuren, and E. Dusseldorp (2014). Recursive partitioning for missing data imputation in the presence of interaction effects. *Computational Statistics & Data Analysis* 72, 92–104.
- DoubleML developers (2025). Causal machine learning with TabPFN (DoubleML example gallery). https://docs.doubleml.org/stable/examples/learners/py_tabpfn.html.

- Epstein, L., W. M. Landes, and R. A. Posner (2010). Inferring the winning party in the Supreme Court from the pattern of questioning at oral argument. *The Journal of Legal Studies* 39(2), 433–467.
- Erickson, N., L. Purucker, A. Tschalzev, D. Holzmüller, P. M. Desai, D. Salinas, and F. Hutter (2025). TabArena: A living benchmark for machine learning on tabular data. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*.
- Farrell, M. H., T. Liang, and S. Misra (2021). Deep neural networks for estimation and inference. *Econometrica* 89(1), 181–213.
- Gelman, A. (2016). Why I prefer 50% rather than 95% intervals. Statistical Modeling, Causal Inference, and Social Science (blog), 5 November, <https://statmodeling.stat.columbia.edu/2016/11/05/why-i-prefer-50-to-95-intervals/>.
- Green, D. P. and H. L. Kern (2012). Modeling heterogeneous treatment effects in survey experiments with Bayesian additive regression trees. *Public Opinion Quarterly* 76(3), 491–511.
- Grimmer, J., S. Messing, and S. J. Westwood (2017). Estimating heterogeneous treatment effects and the effects of heterogeneous treatments with ensemble methods. *Political Analysis* 25(4), 413–434.
- Grinsztajn, L., K. Flöge, O. Key, F. Birkel, P. Jund, B. Roof, B. Jäger, D. Safaric, S. Alessi, A. Hayler, M. Manium, R. Yu, F. Jablonski, S. B. Hoo, A. Garg, J. Robertson, M. Bühler, V. Moroshan, L. Purucker, C. Cornu, L. C. Wehrhahn, A. Bonetto, B. Schölkopf, S. Gambhir, N. Hollmann, and F. Hutter (2025). TabPFN-2.5: Advancing the state of the art in tabular foundation models. arXiv:2511.08667.
- Grinsztajn, L., K. Flöge, O. Key, F. Birkel, P. Jund, B. Roof, M. Manium, S. B. Hoo, M. Bühler, A. Garg, et al. (2026). TabPFN-3: Technical report. arXiv:2605.13986.
- Grinsztajn, L., E. Oyallon, and G. Varoquaux (2022). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*. arXiv:2207.08815.
- Hill, D. W. and Z. M. Jones (2014). An empirical evaluation of explanations for state repression. *American Political Science Review* 108(3), 661–687.
- Hollmann, N., S. Müller, K. Eggenberger, and F. Hutter (2023). TabPFN: A transformer that solves small tabular classification problems in a second. In *International Conference on Learning Representations (ICLR)*. arXiv:2207.01848.
- Hollmann, N., S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter (2025). Accurate predictions on small data with a tabular foundation model. *Nature* 637(8045), 319–326.
- Honaker, J., G. King, and M. Blackwell (2011). Amelia II: A program for missing data. *Journal of Statistical Software* 45(7), 1–47.
- Hünemund, P., B. Louw, and I. Caspi (2023). Double machine learning and automated confounder selection: A cautionary tale. *Journal of Causal Inference* 11(1), 20220078.
- Jäger, B., N. Erickson, L. Grinsztajn, et al. (2026). TabPFN-3.5: Technical report. arXiv:2609.17895.

- Katz, D. M., M. J. Bommarito, and J. Blackman (2017). A general approach for predicting the behavior of the Supreme Court of the United States. *PLoS ONE* 12(4), e0174698.
- Kaufman, A. R., P. Kraft, and M. Sen (2019). Improving Supreme Court forecasting using boosted decision trees. *Political Analysis* 27(3), 381–387.
- Kennedy, E. H. (2024). Semiparametric doubly robust targeted double machine learning: A review. In E. Laber, B. Chakraborty, E. E. M. Moodie, T. Cai, and M. van der Laan (Eds.), *Handbook of Statistical Methods for Precision Medicine*, pp. 207–236. Chapman and Hall/CRC.
- King, G., J. Honaker, A. Joseph, and K. Scheve (2001). Analyzing incomplete political science data: An alternative algorithm for multiple imputation. *American Political Science Review* 95(1), 49–69.
- King, G., M. Tomz, and J. Wittenberg (2000). Making the most of statistical analyses: Improving interpretation and presentation. *American Journal of Political Science* 44(2), 347–361.
- Kropko, J., B. Goodrich, A. Gelman, and J. Hill (2014). Multiple imputation for continuous and categorical data: Comparing joint multivariate normal and conditional approaches. *Political Analysis* 22(4), 497–519.
- Ma, Y., D. Frauen, E. Javurek, and S. Feuerriegel (2025). Foundation models for causal inference via prior-data fitted networks. arXiv:2506.10914.
- Machlanski, D., S. Samothrakis, and P. Clarke (2025). The challenges of hyperparameter tuning for accurate causal effect estimation. arXiv:2303.01412v2.
- Martin, A. D., K. M. Quinn, T. W. Ruger, and P. T. Kim (2004). Competing approaches to predicting Supreme Court decision making. *Perspectives on Politics* 2(4), 761–767.
- Melnichuk, V., V. Balazadeh, S. Feuerriegel, and R. G. Krishnan (2026). Frequentist consistency of prior-data fitted networks for causal inference. In *Proceedings of the 43rd International Conference on Machine Learning*, Volume 306 of *Proceedings of Machine Learning Research*. PMLR.
- Montgomery, J. M. and S. Olivella (2018). Tree-based models for political science data. *American Journal of Political Science* 62(3), 729–744.
- Muchlinski, D., D. Siroky, J. He, and M. Kocher (2016). Comparing random forest with logistic regression for predicting class-imbalanced civil war onset data. *Political Analysis* 24(1), 87–103.
- Müller, S., N. Hollmann, S. Pineda Arango, J. Grabocka, and F. Hutter (2022). Transformers can do Bayesian inference. In *International Conference on Learning Representations (ICLR)*. arXiv:2112.10510.
- Nagler, T. (2023). Statistical foundations of prior-data fitted networks. In *Proceedings of the 40th International Conference on Machine Learning*, Volume 202 of *Proceedings of Machine Learning Research*, pp. 25660–25676. PMLR.
- Neunhoffer, M. (2026a). tabfound: Tabular foundation models in pure R torch. R package version 0.0.0.9000, <https://github.com/mneunhoe/tabfound>.
- Neunhoffer, M. (2026b). tabfound.infer: Statistical inference with tabular foundation models. R package version 0.0.0.9000, included in the replication materials.

- Neunhoeffler, M. and S. Sternberg (2019). How cross-validation can go wrong and what to do about it. *Political Analysis* 27(1), 101–106.
- Qu, J., D. Holzmüller, G. Varoquaux, and M. Le Morvan (2025). TabICL: A tabular foundation model for in-context learning on large data. In *Proceedings of the 42nd International Conference on Machine Learning*, Volume 267 of *Proceedings of Machine Learning Research*, pp. 50817–50847. PMLR.
- Robertson, J., A. Reuter, S. Guo, N. Hollmann, F. Hutter, and B. Schölkopf (2025). Do-PFN: In-context learning for causal effect estimation. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*.
- Rubin, D. B. (1987). *Multiple Imputation for Nonresponse in Surveys*. Wiley Series in Probability and Statistics. New York: John Wiley & Sons.
- Seaman, S. R., J. W. Bartlett, and I. R. White (2012). Multiple imputation of missing covariates with non-linear effects and interactions: an evaluation of statistical methods. *BMC Medical Research Methodology* 12, 46.
- Spaeth, H. J., L. Epstein, A. D. Martin, J. A. Segal, T. J. Ruger, and S. C. Benesh (2025). Supreme Court Database, version 2025 release 01. <https://scdb.la.psu.edu/data/2025-release-01/>. Accessed 23 September 2026.
- Stoetzer, L. F., M. Neunhoeffler, T. Gschwend, S. Munzert, and S. Sternberg (2019). Forecasting elections in multiparty systems: A Bayesian approach combining polls and fundamentals. *Political Analysis* 27(2), 255–262.
- van Buuren, S. and K. Groothuis-Oudshoorn (2011). mice: Multivariate imputation by chained equations in R. *Journal of Statistical Software* 45(3), 1–67.
- van der Laan, M. J., E. C. Polley, and A. E. Hubbard (2007). Super learner. *Statistical Applications in Genetics and Molecular Biology* 6(1), Article 25.
- von Hippel, P. T. (2009). How to impute interactions, squares, and other transformed variables. *Sociological Methodology* 39(1), 265–291.
- Zhang, X., D. C. Maddix, J. Yin, N. Erickson, A. F. Ansari, B. Han, S. Zhang, L. Akoglu, C. Faloutsos, M. W. Mahoney, C. Hu, H. Rangwala, G. Karypis, and B. Wang (2025). Mitra: Mixed synthetic priors for enhancing tabular foundation models. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*.

Appendix

A Software and Replication

The packages. All models in this paper run natively in R on the `torch` backend, without Python, environment management or a GPU. This can matter for researchers who work inside secure environments that hold administrative data. The R package `tabfound` (Neunhoeffer, 2026a) loads TabPFN v2.5, v2.6, v3 and v3.5, TabICL and Mitra through one interface: `tabular_regressor()` and `tabular_classifier()` load a model, `fit()` stores the data as context, and `predict()`, `predict_proba()` and `predict_quantiles()` return point predictions, class probabilities and quantiles of the predictive distribution. The model weights are downloaded once and carry their developers’ licenses, which differ between versions and are listed by `list_models()`. The TabPFN v2.5 runs in this paper used `tabfound`’s predecessor. It agrees with the reference Python implementation to floating-point precision on CPU when the ensemble configurations are generated as in Python, and the package’s regression tests check this agreement.

Imputation. `tabfound_impute()` creates multiply imputed datasets by chained draws from the predictive distribution, with the regression head for continuous and the classification head for categorical variables, and returns them in the formats that `mice` and `Amelia` use for pooling. A `mice` method, `mice.impute.tabfound()`, lets researchers use TabPFN for some variables and other methods for the rest.

Inference. The add-on package `tabfound.infer` (Neunhoeffer, 2026b) contains the estimators of Section 6. `dml_plr()` and `dml_irm()` implement cross-fitted double machine learning for the partially linear and the interactive model. They use TabPFN nuisances by default and accept any learner as a plain R function. Both return the effect, its standard error and the out-of-fold nuisance predictions, so the diagnostics that Bach et al. (2024) recommend can be reported.

Replication. The replication materials contain the analysis data of every exhibit, the simulation scripts with their seeds, and one script that writes every table and figure and recomputes every number in the text against a ledger of the stated values. The Monte Carlo runs are data-intensive (hours of CPU time each), so their results are included as data files and the scripts that produced them are included for rerunning. Where a result depends on a random component (fold assignment, imputation draws), the scripts fix the seed. Everything in this paper ran on CPUs, without a GPU.

B Supreme Court Forecasting: Details

B.1 The Forward Test

The data of the forward test are all argued cases of the 2005 to 2024 terms with an outcome in the 2025 release of the Supreme Court Database, and the outcome is whether the petitioner wins. For each test term from 2015 to 2024, every model is trained on all cases from the 2005 term to the term before and predicts the test term. Every model receives the same training and test rows, and

Table B1. Forecasts of the Future Terms by Feature Set and Model

Model	All 55 features			Argument-day features			Seat-free features		
	Acc.	Log loss	ECE	Acc.	Log loss	ECE	Acc.	Log loss	ECE
Petitioner always wins	67.2	0.634	—	67.2	0.634	—	67.2	0.634	—
KKS, published configuration	70.0	0.671	0.174	69.7	0.673	0.171	63.5	0.688	0.191
AdaBoost, software defaults	67.3	0.628	0.116	68.3	0.629	0.124	67.8	0.628	0.127
TabPFN v2.5	72.0	0.576	0.064	71.0	0.582	0.057	71.1	0.592	0.076
TabPFN v3.5	71.5	0.571	0.056	71.3	0.584	0.052	69.7	0.596	0.071
TabICL	71.8	0.572	0.056	70.6	0.578	0.043	70.0	0.598	0.087
Mitra	71.0	0.579	0.048	72.3	0.578	0.048	70.1	0.592	0.052

Note: Each term OT2015–OT2024 forecast from all earlier terms (603 cases). Accuracy in percent (cutoff 0.5); ECE: expected calibration error with ten equal-mass bins. All 55 features: the features of Kaufman et al. (2019), seat-mapped. Argument-day features: the 47 oral-argument features. Seat-free features: 18 features that do not depend on which justices sit (counts summed over the bench, the relative number of lawyers, and the eight Supreme Court Database codes). All foundation models run with their defaults; the petitioner-always row reports the log loss of a constant forecast at the training-period rate.

there is no tuning, no feature selection and no refitting on test data. The primary comparison is TabPFN v2.5 against the published configuration of Kaufman et al. (2019) (averaged over three seeds) on all 55 features, pooled over the ten terms. Its accuracy difference is reported with a paired case-bootstrap confidence interval (2,000 draws), and the log-loss difference in the same way. Secondary arms (TabPFN v3.5, TabICL, Mitra, AdaBoost at its software defaults, and two further feature sets) are all reported, and none was selected by its score.

Features. The published features come from the official oral-argument transcripts, through code that Kaufman et al. (2019) adapted from an earlier forecasting project. I ran the same chain on all official transcripts of the 2005 to 2024 terms. On the cases of the published sample, the rebuilt relative number of lawyers agrees with the archived feature in 99 percent of cases. Because the published features name individual justices, I assign each justice’s features to the justice who later holds the same seat, and a vacant seat is coded as silence, which is how the original code codes a justice who does not speak.

Other models. Table B1 reports every model on three feature sets of the forward test. The two remaining feature sets give the same picture: on the union of the 55 and the seat-free features, TabPFN predicts 70.3 percent of outcomes correctly and the published configuration 68.7 percent, and on the 55 features without the issue-area code, 71.8 against 69.3 percent. The three other foundation models give the same picture as TabPFN v2.5. On the seat-free features, the published configuration falls below the rule that the petitioner always wins.

B.2 What the Seat-Free Forecast Uses

This analysis is exploratory. On the 18 seat-free features and the forward design, I remove groups of features and refit TabPFN. With all 18 features, it predicts 70.3 percent of outcomes correctly with a log loss of 0.595. Without the relative number of lawyers, accuracy falls to 65.3 percent and the log loss rises to 0.643. The lawyers feature alone gives 67.8 percent and a log loss of 0.604, which

is about three quarters of the improvement of the full feature set over a constant forecast (0.634). Without the features that measure how the bench speaks to each side, the log loss improves to 0.580 at an accuracy of 69.3 percent. This suggests that after 2015 the questioning signal adds little beyond the Solicitor General’s participation. The 18-feature accuracy of this analysis (70.3 percent, with the columns in alphabetical order) differs by about one point from the primary run (71.1 percent), which shows that TabPFN’s predictions depend slightly on the order of the columns.

B.3 Notes on the Replication Archive

Before comparing models, I checked the archive of Kaufman et al. (2019) against the paper and its online appendix. I report the findings that bear on the comparison. None of them was unusual practice at the time, and none would be visible without the authors’ archive.

1. The appendix describes a grid search over the model’s tuning parameters. The archived script cross-validates one fixed configuration, and the choice of features was made against the same cross-validated accuracy that is reported. The published accuracy is therefore a post-selection figure.
2. The model boosts ensembles of 10,000 depth-three extremely randomized trees for up to 10,000 rounds. Because each ensemble fits the reweighted training data perfectly, boosting stops after 10 to 14 rounds on the paper’s folds.
3. The ten folds are contiguous blocks of terms, so most folds are predicted from both earlier and later terms. The forward test in Section 5 removes this.
4. The baseline of 68 percent in the paper is the petitioner-win rate among all cases since 2000. In the 697 analysis cases it is 66 percent, which makes the published model’s gain over the baseline larger than reported.
5. The feature labeled `amicus` is the relative number of lawyers who argue for each side. Coding the side of the United States from the transcripts’ lists of appearances, the United States argues on the side with more lawyers in 90 percent of the cases in which the counts differ.

C Double Machine Learning: Details

Data-generating process. The covariates follow the contest of Montgomery and Olivella (2018): two gamma-distributed covariates, three normal covariates, a three-category variable coded as dummies, two correlated normal covariates, and 30 irrelevant normal covariates. The confounding function g is the regime-switching (“complicated”) surface and the treatment equation m is the interactive (“additive and multiplicative”) surface of that contest. Both depend on the first, second and tenth covariate. Each is standardized to mean zero and variance one with moments from one fixed draw of 100,000 observations, so that the confounding is comparable in size to the unit-variance noise.

Then $D = m(X) + v$ and $Y = 0.5 D + g(X) + \varepsilon$ with independent standard normal v and ε , and each replication draws $n = 500$ observations with its own seed.

Estimators without tuning. PLUG-IN fits TabPFN to Y given X on the full sample, computes $\hat{\theta} = \sum_i D_i(Y_i - \hat{g}(X_i)) / \sum_i D_i^2$, and takes its standard error from the corresponding score. It removes the confounders' influence from the outcome only and uses no cross-fitting. OLS-FULL regresses Y on D and all 40 covariates. The DML estimators use five folds, the partialling-out score and the standard error of Chernozhukov et al. (2018). The random forest has 500 trees and the package defaults otherwise.

Tuned learners. Every tuned learner is tuned on the training part of each cross-fitting fold only, separately for $\hat{\ell}$ and $\hat{m}$. The random forest (500 trees) chooses the number of candidate variables at each split from $\{2, 5, 10, 20, 40\}$ and the minimum node size from $\{1, 5, 10, 20\}$ by out-of-bag error. The lasso uses a dictionary of natural cubic splines with five degrees of freedom for each of the 37 continuous covariates, the three dummies, and all 780 pairwise products of the standardized covariates, and it chooses the penalty by ten-fold cross-validation. Gradient boosting (learning rate 0.05, row and column subsampling of 0.8) chooses the tree depth from $\{2, 4, 6\}$ and the number of rounds (up to 1,000, with early stopping) by five-fold cross-validation. The stack combines OLS, the lasso, the random forest and boosting with non-negative weights that sum to one. The weights minimize the squared error of five-fold inner cross-validated predictions, in which each component keeps the hyperparameters tuned on the training part of the fold. All tuned estimators share the same folds within a replication.

Companion designs and model versions. Table C1 reports two easier designs and two other TabPFN versions. In the canonical design of Chernozhukov et al. (2018), the confounding is close to linear, and all adjustment estimators, including OLS-FULL, cover near the nominal rate. DML with TabPFN also does (94.2 and 96.7 percent), with intervals that are about as wide as those of the other adjustment estimators. On the design of Table 2, TabPFN v2.6 and v3 cover at 94.4 and 94.6 percent.

D Imputation Benchmark: Details

Harness. All runs use the benchmark package MIBench with a fixed seed. Each replication draws a complete dataset, deletes values according to the cell's missingness mechanism, passes the same incomplete dataset to every imputation method ($m = 10$ imputations), fits the cell's analysis model to each imputed dataset, and pools the estimates with Rubin's rules. Coverage is the share of replications in which the pooled 95 percent interval contains the true coefficient. The same harness also fits the analysis model to the complete data, which gives the full-data coverage quoted in Section 7. The rejoinder engines of Table 3 were run later with the same seed and number of cores, and their full-data estimates agree with the archived runs in every replication, so all methods see the same datasets.

Methods. Amelia and mice run at their package defaults (for mice, predictive mean matching

Table C1. Companion Designs and Model Versions

Design and estimator	Bias	RMSE	Coverage (MCSE)	CI width
<i>Canonical partially linear design, continuous treatment (500 replications)</i>				
OLS-FULL	−0.002	0.046	94.0 (1.1)	0.180
DML, random forest (defaults)	0.007	0.045	93.6 (1.1)	0.171
DML, TabPFN (defaults)	0.004	0.045	94.2 (1.0)	0.177
<i>Canonical design, binary treatment, average treatment effect (300 replications)</i>				
Difference in means	0.054	0.108	92.3 (1.5)	0.377
OLS-FULL	0.003	0.094	96.0 (1.1)	0.393
DML, random forest (defaults)	−0.004	0.093	95.0 (1.3)	0.375
DML, TabPFN (defaults)	0.004	0.100	96.7 (1.0)	0.416
<i>Montgomery–Olivella design, other TabPFN versions (500 replications)</i>				
DML, TabPFN v2.6	−0.001	0.045	94.4 (1.0)	0.179
DML, TabPFN v3	0.001	0.045	94.6 (1.0)	0.179

Note: True effect 0.5 in every design. Canonical designs: the simulation design of Chernozhukov et al. (2018) as implemented in DoubleML: $n = 500$, 20 correlated covariates, and confounding through logistic transformations of two of them, which a linear model approximates well. The binary-treatment version draws treatment from a logistic model and targets the average treatment effect with the interactive (AIPW) score and propensities trimmed at 0.02 and 0.98. Version rows: the design and datasets of table 2.

for every variable and five iterations). TabPFN imputes each incomplete variable in turn from all others, starting from random draws of the observed values, with three passes through the variables. Continuous variables are drawn from the regression head’s predictive distribution and categorical variables from the classifier’s predicted probabilities.

Cells. The multivariate normal cells follow the simulation design of King et al. (2001): five correlated normal variables, a linear analysis model, and missingness that is completely at random, at random, or not at random. The binary cells have five correlated binary variables and a logistic analysis model, with missingness in two variables that depends on the other three and increases from the first to the third cell. The three interaction cells are described in Section 7. The 40-variable cells use the three surfaces of Section 4 with 19 percent of the values of each variable missing completely at random and a linear main-effects analysis model. The remaining cells reproduce the variable types of Kropko et al. (2014) and their exercise on the vote-choice model of the 2008 ANES, where the truth is the full-data fit and the benchmark is the distance of the pooled estimates from it.

Table D1. Completed Runs by Benchmark Tier and Imputation Method

Benchmark tier	Amelia	mice	TabPFN
Multivariate normal (5 cells, $n = 500$)	all 1,000 runs	all 1,000 runs	all 1,000 runs
Binary variables, logit model (3, $n = 500$)	600 of 600 failed	all 600 runs	all 600 runs
Interaction in the analysis model (3, $n = 1,000$)	all 600 runs	all 600 runs	all 600 runs
Montgomery–Olivella, 40 variables (3, $n = 500$)	did not finish	all 150 runs	all 150 runs
Kropko et al. (2014) variable types (4, $n = 1,000$)	all 800 runs	all 800 runs	all 800 runs
ANES 2008 vote choice (1, $n = 1,442$)	all 50 runs	all 50 runs	all 50 runs

Note: Runs in which the method returned ten imputed datasets that the analysis model could use. Amelia’s run on the 40-variable tier did not finish: its EM algorithm ran for about six hours on the first cell and then stopped with a numerical error, and I did not rerun it. The 40-variable tier and the ANES exercise have 50 replications per cell, the other tiers 200.